Lowering 단계 상세 분석

개요

Lowering 단계는 Cairo 컴파일러에서 HIR(High-level Intermediate Representation)을 MIR(Mid-level Intermediate Representation)로 변환하는 핵심 과정입니다. 이 단계에서는 고수준의 의미론적 표현을 더 단순하고 실행 가능한 형태로 변환하며, 패턴 매칭 해제, 제어 흐름 단순화, 함수 인라이닝 등의 중요한 변환 작업을 수행합니다.

cairo-lang-lowering Crate 구조

주요 모듈 구성

```
cairo-lang-lowering/
├── src/
| ├── lib.rs # 메인 라이브러리 진입점
| ├── db.rs # Salsa 데이터베이스 쿼리 정의
| ├── objects.rs # 핵심 데이터 구조 정의
| ├── lower/ # 메인 lowering 로직
| | ├── mod.rs # 함수 lowering 메인 로직
| | ├── context.rs # Lowering 컨텍스트 관리
| | ├── block_builder.rs # 블록 구성 도구
| | ├── lower_match.rs # 패턴 매칭 lowering
| | ├── lower_if.rs # 조건문 lowering
```

```
| | └── generators.rs # 코드 생성 유틸리티
| ├── optimizations/ # 최적화 패스들
| ├── inline/ # 함수 인라이닝
| ├── specialization.rs # 함수 특수화
| └── borrow_check/ # 소유권 검사
```

핵심 데이터 구조

Lowered 구조체

```rust
pub struct Lowered<'db> {
/// 진단 정보
pub diagnostics: Diagnostics<'db, LoweringDiagnostic<'db>>,
/// 함수 시그니처
pub signature: Signature<'db>,
/// 변수 아레나
pub variables: VariableArena<'db>,
/// 블록 아레나
pub blocks: Blocks<'db>,
/// 함수 매개변수
pub parameters: Vec<VariableId>,
}
```

Variable 구조체

```rust
pub struct Variable<'db> {
/// 의미론적 타입
pub ty: semantic::TypeId<'db>,
/// 위치 정보
pub location: LocationId<'db>,
/// 타입 정보 (copyable, droppable 등)
pub info: TypeInfo<'db>,
}
```

Statement 열거형

```rust
pub enum Statement<'db> {
Const(StatementConst<'db>),
Call(StatementCall<'db>),
StructConstruct(StatementStructConstruct<'db>),
StructDestructure(StatementStructDestructure<'db>),
EnumConstruct(StatementEnumConstruct<'db>),
Snapshot(StatementSnapshot<'db>),
Desnap(StatementDesnap<'db>),
}
```

HIR에서 MIR로의 변환 과정

1. 함수 Lowering 메인 플로우

```
pub fn lower_semantic_function<'db>(
db: &'db dyn LoweringGroup,
semantic_function_id: defs::ids::FunctionWithBodyId<'db>,
) -> Maybe<MultiLowering<'db>>
```

변환 단계:

1. **컨텍스트 초기화**: `EncapsulatingLoweringContext` 생성
2. **시그니처 변환**: 의미론적 시그니처를 lowered 시그니처로 변환
3. **매개변수 처리**: 함수 매개변수를 lowered 변수로 변환
4. **블록 표현식 lowering**: 함수 본문을 블록 단위로 처리
5. **다중 lowering 생성**: 메인 함수와 생성된 함수들(클로저, 루프 등) 포함

2. 표현식 Lowering

LoweredExpr 열거형

```
pub enum LoweredExpr<'db> {
AtVariable(VarUsage<'db>),
Tuple { exprs: Vec<LoweredExpr<'db>>, location: LocationId<'db> },
FixedSizeArray { ty: TypeId<'db>, exprs: Vec<LoweredExpr<'db>>, location: Locati
ExternEnum(LoweredExprExternEnum<'db>),
MemberPath(ExprVarMemberPath<'db>, LocationId<'db>),
Snapshot { expr: Box<LoweredExpr<'db>>, location: LocationId<'db> },
```

```
}
```

변환 과정:

- **변수 참조**: 직접적인 변수 사용
- **튜플/구조체**: 구성 요소들의 재귀적 lowering
- **외부 함수 호출**: 열거형 결과를 가진 extern 함수 처리
- **멤버 경로**: 구조체/튜플 멤버 접근
- **스냅샷**: 불변 참조 생성

3. 블록 구성

BlockBuilder 패턴

```
pub struct BlockBuilder<'db> {
pub block_id: BlockId,
pub statements: Vec<Statement<'db>>,
// 변수 바인딩과 참조 관리
}
```

기능:

- **문장 추가**: 순차적 문장 구성
- **변수 관리**: 스코프 내 변수 바인딩
- **제어 흐름**: 분기와 점프 처리
- **블록 병합**: 여러 블록의 합성

패턴 매칭 해제 과정

1. 패턴 매칭 변환 전략

단일 패턴 처리

```
fn lower_single_pattern<'db>(
ctx: &mut LoweringContext<'db, '_>,
builder: &mut BlockBuilder<'db>,
pattern_id: semantic::PatternId,
lowered_expr: LoweredExpr<'db>,
) -> LoweringResult<'db, ()>
```

지원되는 패턴:

- **변수 패턴**: 직접 변수 바인딩
- **구조체 패턴**: 구조체 해체 (destructuring)
- **튜플 패턴**: 튜플 요소 분해
- **열거형 패턴**: 변형 매칭과 값 추출

매치 표현식 처리

```
pub enum MatchInfo<'db> {
Enum(MatchEnumInfo<'db>), // 열거형 매칭
Extern(MatchExternInfo<'db>), // 외부 함수 결과 매칭
Value(MatchEnumValue<'db>), // 값 기반 매칭
```

```
}
```

2. 열거형 매칭 해제

변환 과정:

1. **변형 추출**: 열거형의 모든 변형 분석
2. **분기 생성**: 각 변형에 대한 별도 블록 생성
3. **값 바인딩**: 변형 내부 값을 새 변수에 바인딩
4. **블록 병합**: 모든 분기의 결과 통합

```
pub struct MatchEnumInfo<'db> {
pub concrete_enum_id: ConcreteEnumId<'db>,
pub input: VarUsage<'db>,
pub arms: Vec<MatchArm<'db>>,
pub location: LocationId<'db>,
}
```

3. 구조체 해체

```
pub struct StatementStructDestructure<'db> {
pub input: VarUsage<'db>,
pub outputs: Vec<VariableId>,
}
```

처리 방식:

- **필드 추출**: 구조체의 각 필드를 개별 변수로 분리
- **타입 보존**: 각 필드의 타입 정보 유지
- **위치 추적**: 원본 구조체와 필드들의 위치 관계 보존

제어 흐름 단순화 메커니즘

1. 조건문 변환

if 표현식 처리

```
pub fn lower_expr_if<'db>(
ctx: &mut LoweringContext<'db, '_>,
builder: &mut BlockBuilder<'db>,
expr_if: semantic::ExprIf<'db>,
) -> LoweringResult<'db, LoweredExpr<'db>>
```

변환 단계:

1. **조건 평가**: 조건식을 boolean 변수로 변환
2. **분기 생성**: true/false 각각에 대한 블록 생성
3. **매치 변환**: boolean 열거형에 대한 매치로 변환
4. **결과 통합**: 두 분기의 결과를 통합

2. 루프 변환

while 루프 처리

```
pub fn lower_while_loop<'db>(
```

```
ctx: &mut LoweringContext<'db, '_>,
builder: &mut BlockBuilder<'db>,
loop_expr: semantic::ExprWhile<'db>,
loop_expr_id: semantic::ExprId,
) -> LoweringResult<'db, LoweredExpr<'db>>
```

변환 방식:

- **재귀 함수 생성**: 루프를 재귀 함수 호출로 변환
- **조건 검사**: 각 반복에서 조건 평가
- **탈출 처리**: break/continue 문의 적절한 변환

for 루프 처리

```
pub fn lower_for_loop<'db>(
ctx: &mut LoweringContext<'db, '_>,
builder: &mut BlockBuilder<'db>,
loop_expr: semantic::ExprFor<'db>,
loop_expr_id: semantic::ExprId,
) -> LoweringResult<'db, LoweredExpr<'db>>
```

Iterator 기반 변환:

1. **Iterator 생성**: `into_iter()` 호출
2. **next() 호출**: 각 반복에서 다음 요소 획득
3. **Option 매칭**: Some/None에 따른 분기 처리

4. **패턴 바인딩**: 루프 변수에 값 바인딩

3. 에러 전파 처리

```
pub fn lower_expr_error_propagate<'db>(
ctx: &mut LoweringContext<'db, '_>,
builder: &mut BlockBuilder<'db>,
expr: semantic::ExprPropagateError<'db>,
) -> LoweringResult<'db, LoweredExpr<'db>>
```

? 연산자 변환:

- **Result/Option 매칭**: 성공/실패 케이스 분리
- **조기 반환**: 에러 케이스에서 함수 조기 종료
- **값 추출**: 성공 케이스에서 내부 값 추출

함수 인라이닝 전략

1. 인라이닝 결정 로직

```
pub fn priv_should_inline<'db>(
db: &'db dyn LoweringGroup,
function_id: ConcreteFunctionWithBodyId<'db>,
) -> Maybe<bool>
```

결정 요소:

- **함수 크기**: 문장 수와 복잡도 기반 임계값
- **호출 빈도**: 함수 호출 패턴 분석
- **순환 참조**: 재귀 함수는 인라이닝 제외
- **명시적 지시**: `#[inline(always)]`, `#[inline(never)]` 속성

2. 인라이닝 구현

변수 재매핑

```
pub struct VarRemapping<'db> {
pub remapping: OrderedHashMap<VariableId, VarUsage<'db>>,
}
```

인라이닝 과정:

1. **함수 본문 복사**: 호출될 함수의 lowered 표현 복사
2. **변수 재매핑**: 함수 내부 변수를 호출 컨텍스트로 매핑
3. **매개변수 바인딩**: 인수를 매개변수 변수에 바인딩
4. **반환값 처리**: 함수 반환을 호출 지점 변수로 연결
5. **블록 통합**: 인라인된 블록들을 호출 함수에 통합

3. 인라이닝 최적화

가중치 기반 결정

```
pub trait InlineWeight<'db> {
fn lowered_weight(&self, lowered: &Lowered<'db>) -> isize;
```

```
}
```

가중치 계산:

- **문장 타입별 가중치**: 각 문장 유형의 비용 계산
- **호출 깊이**: 중첩된 호출의 복잡도 고려
- **메모리 사용량**: 변수 수와 타입 크기 고려

특수화 (Specialization)

1. 함수 특수화 개념

```
pub struct SpecializedFunction<'db> {
pub base: ConcreteFunctionWithBodyId<'db>,
pub args: Vec<Option<SpecializationArg<'db>>>,
}
```

특수화 대상:

- **상수 매개변수**: 컴파일 타임에 알려진 값들
- **빈 배열**: 크기가 0인 배열 매개변수
- **구조체 상수**: 상수 필드를 가진 구조체

2. 특수화 인수 타입

```
pub enum SpecializationArg<'db> {
Const(ConstValueId<'db>),
```

```
EmptyArray(TypeId<'db>),

Struct(Vec<SpecializationArg<'db>>),

}
```

3. 특수화 변환 과정

변환 단계:

1. **상수 인수 식별**: 컴파일 타임 상수 매개변수 찾기
2. **특수화된 함수 생성**: 상수 값이 하드코딩된 새 함수 생성
3. **호출 지점 교체**: 원본 함수 호출을 특수화된 함수 호출로 교체
4. **최적화 기회**: 상수 폴딩과 데드 코드 제거 활성화

최적화 패스

1. 주요 최적화 기법

상수 폴딩 (Constant Folding)

```
pub fn const_folding_optimization<'db>(

db: &'db dyn LoweringGroup,

lowered: &mut Lowered<'db>,

) -> Maybe<()>
```

최적화 대상:

- **산술 연산**: 컴파일 타임 계산 가능한 연산
- **논리 연산**: boolean 상수 연산

- **비교 연산**: 상수 간 비교

공통 부분식 제거 (CSE)

```
pub fn cse_optimization<'db>(
db: &'db dyn LoweringGroup,
lowered: &mut Lowered<'db>,
) -> Maybe<()>
```

제거 대상:

- **중복 계산**: 동일한 표현식의 반복 계산
- **변수 재사용**: 동일한 값을 가진 변수들 통합

데드 코드 제거

```
pub fn trim_unreachable_optimization<'db>(
db: &'db dyn LoweringGroup,
lowered: &mut Lowered<'db>,
) -> Maybe<()>
```

제거 대상:

- **도달 불가능한 블록**: 실행되지 않는 코드 블록
- **사용되지 않는 변수**: 정의되었지만 사용되지 않는 변수

2. 최적화 전략

단계별 최적화

```
pub enum LoweringStage {
Monomorphized, // 단형화 후
PreOptimizations, // 기본 최적화 전
PostBaseline, // 기본 최적화 후
Final, // 최종 최적화 후
}
```

각 단계별 처리:

- **Monomorphized**: 제네릭 해결과 구체화
- **PreOptimizations**: 가스 추가, 패닉 처리, 소멸자 추가
- **PostBaseline**: 기본 최적화 패스 적용
- **Final**: 최종 최적화와 정리

에러 처리와 진단

1. Lowering 에러 타입

```
pub enum LoweringFlowError<'db> {
Failed(DiagnosticAdded),
Panic(VarUsage<'db>, LocationId<'db>),
Return(VarUsage<'db>, LocationId<'db>),
Match(MatchInfo<'db>),
}
```

2. 진단 정보

```
pub enum LoweringDiagnosticKind<'db> {
UnsupportedPattern,
VariableNotFound { var_id: semantic::VarId<'db> },
MatchError(MatchError<'db>),
Unreachable { block_end_ptr: SyntaxStablePtrId<'db> },
// ... 기타 진단 타입들
}
```

성능 고려사항

1. 메모리 관리

- **Arena 할당**: 변수와 블록의 효율적인 메모리 관리
- **참조 카운팅**: 공유 데이터 구조의 최적화
- **지연 평가**: 필요시에만 계산하는 전략

2. 컴파일 시간 최적화

- **캐싱**: Salsa 기반 쿼리 캐싱
- **점진적 컴파일**: 변경된 부분만 재컴파일
- **병렬 처리**: 독립적인 함수들의 병렬 lowering

3. 코드 품질

- **타입 안전성**: 강타입 시스템을 통한 오류 방지

- **불변성**: 가능한 한 불변 데이터 구조 사용
- **명확한 소유권**: Rust의 소유권 시스템 활용

결론

Lowering 단계는 Cairo 컴파일러에서 고수준 의미론적 표현을 실행 가능한 중간 표현으로 변환하는 핵심 과정입니다. 패턴 매칭 해제, 제어 흐름 단순화, 함수 인라이닝, 그리고 다양한 최적화 기법을 통해 효율적이고 최적화된 코드를 생성합니다. 이 과정에서 생성된 MIR은 다음 단계인 Sierra 생성의 입력으로 사용되어 최종적으로 실행 가능한 코드로 변환됩니다.